

Some important terms

Assembler: An assembler is a program that takes basic computer instructions and converts them into a pattern of bits that the computer's processor can use to perform its basic operations. The assembler creates object code by translating assembly instruction mnemonics into opcodes and by resolving symbolic names for memory locations and other entities.

Compiler: A compiler is a special program that processes statements written in a particular programming language and turns them into machine language or 'code' that a computer's processor uses. The reason for transforming source code is to create an executable program for the target.

Debugger: A debugger is a computer program that is used to test and debug other programs (the 'target' program).

Linkers: In computer science, a linker or link editor is a program that takes one or more objects generated by a compiler and combines them into a single executable program.

IDE: IDE (integrated development environment) is an integrated software tools suite that consists of source code editor, compiler and debugger.

Simulator: A simulator is a program that imitates the real phenomenon in an artificial environment.

SysML: The SysML (systems modeling language) is a general-purpose modeling language for systems engineering applications that is defined as a dialect (Profile) of UML 2. It supports the specification, analysis, design, verification and validation of a broad range of systems and systems-of-systems.

UML: UML (unified modeling language) is an industry standard modeling language with a rich graphical notation, and comprehensive set of diagrams and elements.

"Compilers and debuggers always need to have some knowledge of the target application. Run-time libraries also need to be processor-specific," expresses Almstrom. "Code generation and design tools can be platform independent," he adds. Further expressing his views, he says, "Between target-specific and platform independent, there is an interesting area of 'lightly customised,' where a common tool takes a small amount of work to adapt it to different platforms."

John from IBM says, "The tools that are closest to the hardware platform need to be more concerned with the target platform. These are testing tools. But with virtualisation, it is possible to have design tools running on one platform and run tests on the same computer in a virtual environment." Although tools still may not be platform-specific, virtualisation may enable early validation of designs without waiting for availability of the real target."

Expressing his views from customer standpoint, Bhandari from Intel says, "As a software vendor, there is a need to support multi-platform solutions. You need to provide the flexibility to customers to switch between

platforms."

Says John, "With rising capabilities of the Internet, more functionality is moving towards a web-based client server type model that is really platform independent."

Catch-22 in tools selection

Software tools' selection depends on the core architecture of your embedded processor. Embedded vendors offer a variety of tools for the entire embedded product development cycle. "As you decide as to what you want to develop, you can choose hardware chipsets and kernels. Depending on that, you need to select tools to proceed further. However, the choice may be limited due to inventory, previous volume commitments and existing product designs," expresses Almstrom.

The next critical stage that further complicates the matter is availability of tools. "Availability of software tools depends on the choice of base platform. If you choose a proprietary platform then you need to use the tools that are either developed in-house or provided by the community that supports it," says Bhandari. Expressing his views on multi-platform tools, he says there

is a need to provide extended support to multi-platform tools, as it provides flexibility to developers to switch between platforms.

Another key factor that influences the selection of tools is the price. "Sophisticated tools always carry higher price. A developer wants to buy tools at lower price. Also, many tools available at lower price offer limited features and support that may not satisfy the complete requirements," says Gurunatham, ESA.

Changing trends

As systems get complex, designers demand more user-friendly tools. We have seen more emphasis on the "graphical tools" to experience the actual look and feel of the product at every stage. Most of the software tools have graphical representation starting from design, simulation, software development, testing and validation. The results from certain tools are also represented graphically.

There are a number of open source tools available. "Some tools like GCC, OpenCV are quite popular," shares Bhandari. There are cross-compilers, middleware stacks, RTOSes that are open source now, but support needs to be provided by the community.

Almstrom says, "The advent of Eclipse as a standard IDE, which is open source, is a strong encouragement to develop tools to work on many host platforms. With respect to targets, the market has always favoured either platform-specific tools (especially if silicon manufacturers sponsor the development of the tools) or multi-platform tools that are segmented by target market."

To sum it up, there are plethora of tools and operating systems, both proprietary and open source, platform specific and multi-platform, integrated and segregated, but you have to pick the one that excels in your application. Selecting a proper tool may be hard but a wrong tool may hit you back in the form of bug and missed deadlines! ●

The author is senior journalist (technology) at EYF